

1. Pourquoi des frameworks côté client comme Angular ont-ils été introduits ?

* Les frameworks côté client permettent de développer des applications Web avancées comme Single-Page-Application.

* Ils ont facilité la vie des développeurs en gérant la séparation des préoccupations et en divisant le code en plus petits éléments

2. Comment fonctionne une application Angular ?

Chaque application Angular consiste en un fichier nommé **angular.json** . Ce fichier contiendra toutes les configurations de l'application. Lors de la création de l'application, le générateur consulte ce fichier pour trouver le point d'entrée de l'application. Voici une image du fichier angular.json

```
"build": {
  "builder": "@angular-devkit/build-angular:browser",
  "options": {
    "outputPath": "dist/angular-starter",
    "index": "src/index.html",
    "main": "src/main.ts",
    "polyfills": "src/polyfills.ts",
    "tsConfig": "tsconfig.app.json",
    "aot": false,
    "assets": [
      "src/favicon.ico",
      "src/assets"
    ],
    "styles": [
      "../node_modules/@angular/material/prebuilt-themes/deeppurple-amber.css",
      "src/style.css"
    ]
  }
}
```

Le fichier main.ts crée un environnement de navigateur pour l'exécution de l'application et, parallèlement à cela, il appelle également une fonction appelée **bootstrapModule** , qui démarre l'application

```
import { platformBrowserDynamic } from '@angular/platform-browser-dynamic';
```

```
platformBrowserDynamic().bootstrapModule(AppModule)
```

L'AppModule est déclaré dans le fichier app.module.ts. Ce module contient les déclarations de tous les composants.

Après la déclaration des composants dans le fichier app.module.ts, on passe à l composant qui été créé :

```
import { Component } from '@angular/core';

@Component({
  selector: 'app-root',
  templateUrl: './app.component.html',
  styleUrls: ['./app.component.css']
})
export class AppComponent {
  title = 'angular';
}
```

Sélecteur - utilisé pour accéder au composant

Template/TemplateURL - contient le code HTML du composant

StylesURL - contient des feuilles de style spécifiques aux composants

* Après cela, Angular appelle le fichier **index.html** . Ce fichier appelle par conséquent le composant racine qui est **app-root** . Le composant racine est défini dans **app.component.ts** . Voici à quoi ressemble le fichier index.html

3. Quels sont certains des avantages d'Angular par rapport aux autres frameworks ?

- **Fonctionnalités fournies prêtes à l'emploi** - Angular fournit un certain nombre de fonctionnalités intégrées telles que le routage, la gestion d'état, la bibliothèque rxjs et les services http dès la sortie de la boîte. Cela signifie qu'il n'est pas nécessaire de rechercher les caractéristiques susmentionnées séparément. Ils sont tous pourvus d'angles.
- **Interface utilisateur déclarative** - Angular utilise HTML pour rendre l'interface utilisateur d'une application. HTML est un langage déclaratif et est beaucoup plus facile à utiliser que JavaScript.
- **Support Google à long terme** - Google a annoncé un support à long terme pour Angular. Cela signifie que Google prévoit de s'en tenir à Angular et de développer davantage son écosystème.

4. Quels sont les avantages d'Angular par rapport à React ?

Angulaire	Réagir
Angular prend en charge la liaison de données bidirectionnelle ainsi que les données modifiables.	React ne prend en charge que la liaison de données unidirectionnelle et immuable.
Le plus grand avantage d'Angular est qu'il permet l'injection de dépendances.	React nous permet de l'accomplir nous-mêmes ou avec l'aide d'une bibliothèque tierce.
Angular peut être utilisé à la fois dans le développement mobile et Web.	React ne peut être utilisé que dans le développement de l'interface utilisateur.
Angular propose une large gamme d'outils, de bibliothèques, de frameworks, de plugins, etc., qui rendent le développement plus rapide et plus amusant.	Dans React, nous pouvons utiliser des bibliothèques tierces pour toutes les fonctionnalités.
Angular utilise Typescript.	Réagir utilise Javascript.

5. Lister les différences entre AngularJS et Angular ?

- **Architecture**
 - AngularJS utilise l'architecture MVC ou Model-View-Controller, où le modèle contient la logique métier, le contrôleur traite les informations et la vue affiche les informations présentes dans le modèle.
 - Angular remplace les contrôleurs par des composants. Les composants ne sont rien d'autre que des directives avec un modèle prédéfini.
- **Langue**
 - AngularJS utilise le langage JavaScript, qui est un langage typé dynamiquement.
 - Angular utilise le langage TypeScript, qui est un langage typé statiquement et est un sur-ensemble de JavaScript. En utilisant un langage typé statiquement, Angular offre de meilleures performances tout en développant des applications plus volumineuses.
- **Assistance mobile**
 - AngularJS ne fournit pas de support mobile.
 - Angular est pris en charge par tous les navigateurs mobiles populaires.
- **Structure**
 - Lors du développement d'applications plus volumineuses, le processus de maintenance du code devient fastidieux dans le cas d'AngularJS.
 - Dans le cas d'Angular, il est plus facile de maintenir le code pour des applications plus importantes car il fournit une meilleure structure.
- **Syntaxe des expressions**
 - Lors du développement d'une application AngularJS, un développeur doit se souvenir de la bonne directive ng pour lier un événement ou une propriété. Alors que dans Angular, la liaison de propriété est effectuée à l'aide de l'attribut "[]" et la liaison d'événement est effectuée à l'aide de l'attribut "()".

6. En quoi les expressions angulaires sont-elles différentes des expressions JavaScript ?

* les expressions angulaires sont évaluées par rapport à un objet de portée **locale** , tandis que les expressions JavaScript sont comparées à un objet de fenêtre **global** . Comprenons mieux cela avec un exemple :

```
import { Component, OnInit } from '@angular/core';

@Component({
  selector: 'app-test',
  template: `
    <h4>{{message}}</h4>
  `,
  styleUrls: ['./test.component.css']
})
export class TestComponent implements OnInit {
  message:string = "Hello world";
  constructor() { }

  ngOnInit() {
  }
}
```

* La différence suivante est la façon dont les expressions angulaires gèrent **null** et **undefined**

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>JavaScript Test</title>
  </head>
  <body>
    <div id="foo"><div>
  </body>
  <script>
    'use strict';
    let bar = {};
    document.getElementById('foo').innerHTML = bar.x;
  </script>
</html>
```

undefined

```
import { Component, OnInit } from '@angular/core';

@Component({
  selector: 'app-new',
  template: `
    <h4>{{message}}</h4>
  `,
  styleUrls: ['./new.component.css']
})
export class NewComponent implements OnInit {
  message:object = {};
  constructor() { }

  ngOnInit() {
  }
}
```

null

* La différence qui rend les expressions angulaires très bénéfiques est l'utilisation de `pipe` . Angular utilise des `pipe` (appelés `filters` dans AngularJS), qui peuvent être utilisés pour formater les données avant de les afficher

```
import { Component, OnInit } from '@angular/core';

@Component({
  selector: 'app-new',
  template: `
    <h4>{{message | lowercase}}</h4>
  `,
  styleUrls: ['./new.component.css']
})
export class NewComponent implements OnInit {
  message:string = "HELLO WORLD";
  constructor() { }

  ngOnInit() {
  }
}
```

7. Qu'est-ce qu'une application monopage (SPA) ?

* Les applications à page unique sont des applications Web qui ne doivent être chargées qu'une seule fois, avec de nouvelles fonctionnalités consistant en des modifications mineures de l'interface utilisateur

* Il ne charge pas de nouvelles pages HTML pour afficher le contenu de la nouvelle page, mais le génère plutôt dynamiquement

8. Que sont les templates dans Angular ?

Un modèle est une sorte de code HTML qui indique à Angular comment afficher un composant

Il existe deux manières de créer un template dans un composant Angular :

- Modèle en ligne
- Modèle lié

Modèle en ligne : la configuration du modèle du décorateur de composant est utilisée pour spécifier un modèle HTML en ligne pour un composant. Le modèle sera entouré de guillemets simples ou doubles.

Modèle lié : un composant peut inclure un modèle HTML dans un fichier HTML distinct. Comme illustré ci-dessous, l'option `templateUrl` est utilisée pour indiquer le chemin du fichier de modèle HTML.

```
@Component({
  selector: "app-greet",
  template: `<div>
    <h1> Hello {{name}} how are you ? </h1>
    <h2> Welcome to interviewbit ! </h2>
  </div>`
})
```

```
@Component({
  selector: "app-greet",
  templateUrl: "./component.html"
})
```

9. Que sont les directives dans Angular ?

Une directive est une classe dans Angular qui est déclarée avec un décorateur **@Directive** :

1. Directives des composants

Celles-ci forment la classe principale dans les directives. **Au lieu** du décorateur **@Directive**, nous utilisons le décorateur **@Component** pour déclarer ces directives. Ces directives ont une vue, une feuille de style et une propriété `selector`.

2. Directives structurelles

- Ces directives sont généralement utilisées pour manipuler des éléments DOM.
- Chaque directive structurelle est précédée d'un signe ' * '.
- Nous pouvons appliquer ces directives à n'importe quel élément DOM.

3. Directives d'attribut

- Ces directives sont utilisées pour modifier l'apparence et le comportement d'un élément DOM. Comprenons les directives d'attribut en en créant une :

```
<div *ngIf="isReady" class="display_name">
  {{name}}
</div>

<div class="details" *ngFor="let x of details" >
  <p>{{x.name}}</p>
  <p> {{x.address}}</p>
  <p>{{x.age}}</p>
</div>
```

Comment créer une directive personnalisée ? : ng g directive blueBackground

```
import { Directive, ElementRef } from '@angular/core';

@Directive({
  selector: '[appBlueBackground]'
})
export class BlueBackgroundDirective {
  constructor(el: ElementRef) {
    el.nativeElement.style.backgroundColor = "blue";
  }
}
```

Nous pouvons maintenant appliquer la directive ci-dessus à n'importe quel élément DOM :

```
appBlueBackground>Hello World!</p>
```

10. Expliquer les composants, modules et services dans Angular

- **Composants** : les composants sont les blocs de construction de base, qui contrôlent une partie de l'interface utilisateur pour n'importe quelle application.

Un composant est défini à l'aide du décorateur **@Component**. Chaque composant se compose de trois parties, le modèle qui charge la vue du composant, une feuille de style qui définit l'apparence du composant et une classe qui contient la logique métier du composant.

```
ng generate component test ou ng g c test
```

- **Modules** : Un module est un endroit où nous pouvons regrouper des composants, des directives, des services et des canaux. Le module décide si les composants, les directives, etc. peuvent être utilisés par d'autres modules, en exportant ou en masquant ces éléments. Chaque module est défini avec un décorateur **@NgModule**.

```
ng g m test-module
```

Les modules sont de deux types :

- Module racine
- Module de fonctionnalités

Un module racine importe **BrowserModule**, tandis qu'un module de fonctionnalité importe **CommonModule**.

```
import { BrowserModule } from '@angular/platform-browser';
import { NgModule } from '@angular/core';

import { AppComponent } from './app.component';
import { TestComponent } from './test/text.component';

@NgModule({
  declarations: [
    AppComponent,
    TestComponent
  ],
  imports: [
    BrowserModule
  ],
  providers: [],
  bootstrap: [AppComponent]
})
export class AppModule { }
```

```
import { NgModule } from '@angular/core';
import { CommonModule } from '@angular/common';

@NgModule({
  declarations: [],
  imports: [
    CommonModule
  ]
})
export class TestModuleModule { }
```

- **Services**

* Les services sont des objets qui ne sont instanciés qu'une seule fois pendant la durée de vie d'une application. L'objectif principal d'un service est de partager des données, des fonctions avec différents composants d'une application Angular

* Un service est défini à l'aide d'un décorateur **@Injectable**. Une fonction définie à l'intérieur d'un service peut être invoquée à partir de n'importe quel composant ou directive

```
ng g s test-service
```

```
import { Injectable } from '@angular/core';

@Injectable({
  providedIn: 'root'
})
export class TestServiceService {

  constructor() { }

}
```

11. What is the scope?

Dans Angular, une portée est un objet qui fait référence au modèle d'application. C'est un contexte dans lequel des expressions peuvent être exécutées. Ces portées sont regroupées hiérarchiquement, comparable à la structure DOM de l'application. Une portée facilite la propagation de divers événements et la surveillance des expressions

12. What is data binding in Angular?

Il existe quatre types de liaison de données dans Angular :

- Liaison de propriété
- Liaison d'événement
- Interpolation de chaîne
- Liaison de données bidirectionnelle

Liaison de propriété : une méthode de liaison de données est appelée liaison de propriété. Dans la liaison de propriété, nous connectons la propriété d'un élément DOM à un champ qui est une propriété déclarée dans notre code de composant TypeScript. En réalité, Angular transforme l'interpolation de chaîne en liaison de propriété en interne.

Liaison d'événements : à l'aide de la liaison d'événements, vous pouvez répondre aux événements DOM tels que les clics sur les boutons et les mouvements de la souris. Lorsqu'un événement DOM (tel qu'un clic, un changement ou un keyup) se produit, la méthode désignée du composant est appelée.

Interpolation de chaîne : afin d'exporter des données du code TypeScript vers un modèle HTML (vue), l'interpolation de chaîne est une approche de liaison de données à sens unique. Les données du composant sont affichées dans la vue à l'aide de l'expression de modèle entourée d'accolades doubles. La valeur d'une propriété de composant est ajoutée à l'aide de l'interpolation de chaîne

Liaison de données bidirectionnelle : Le partage de données entre une classe de composants et son modèle est appelé liaison de données bidirectionnelle. Si vous modifiez des données dans une zone, elles seront immédiatement recalculées à l'autre extrémité. Cela se produit instantanément et automatiquement, garantissant que le modèle HTML et le code TypeScript sont toujours à jour. La liaison de propriété et la liaison d'événement sont couplées dans une liaison de données bidirectionnelle.

14. What are Decorators and their types in Angular?

Les décorateurs sont des méthodes ou des modèles de conception qui sont étiquetés avec un symbole @ préfixé et précédés d'une classe, d'une méthode ou d'une propriété. Ils permettent de modifier un service, une directive ou un filtre avant son utilisation

Types de décorateurs :

- **Décorateur de méthode** : les décorateurs de méthode, comme leur nom l'indique, sont utilisés pour ajouter des fonctionnalités aux méthodes définies dans notre classe.
- **Décorateur de classe** : Les décorateurs de classe sont les décorateurs de plus haut niveau qui déterminent le but des classes. Ils indiquent à Angular qu'une classe spécifique est un composant ou un module. Et le décorateur nous permet de déclarer cet effet sans avoir à écrire de code dans la classe.
- **Décorateur de paramètres** : les arguments de vos constructeurs de classe sont décorés à l'aide de décorateurs de paramètres.
- **Décorateur de propriété** : Ce sont les deuxièmes types de décorateurs les plus populaires. Ils nous permettent de valoriser certaines propriétés de nos classes

15. What are annotations in Angular?

Ce sont des fonctionnalités de langage qui sont codées en dur. Les annotations sont simplement des métadonnées définies sur une classe pour refléter la bibliothèque de métadonnées

16. What are pure ?

Ce sont des pipelines qui n'emploient que des fonctions pures. En conséquence, un pipe pur n'emploie aucun état interne et la sortie reste constante tant que les paramètres fournis restent constants. Angular appelle le tube uniquement lorsque les paramètres fournis changent. Une seule instance du pipe pur est utilisée dans tous les composants.

17. What are impure Pipes?

Angular appelle un pipe impur pour chaque cycle de détection de changement, indépendamment du changement dans les champs de saisie. Pour chacun de ces pipes, plusieurs instances de pipes sont produites. Les entrées de ces pipes peuvent être modifiées.

Par défaut, tous les pipelines sont purs. Cependant, comme illustré ci-dessous, vous pouvez spécifier des pipes impurs à l'aide de la propriété `pure`.

```
@Pipe({
  name: 'impurePipe',
  pure: false/true
})
export class ImpurePipe {}
```

18. What is Pipe transform Interface in Angular?

Une interface utilisée par les pipes pour accomplir une transformation. Angular appelle la fonction de transformation avec la valeur d'une liaison comme premier argument et tous les arguments comme deuxième paramètre sous forme de liste. Cette interface est utilisée pour implémenter des pipes personnalisés.

```
import { Pipe, PipeTransform } from '@angular/core';
@Pipe({
  name: 'transformpipe'
})
export class TransformPipe implements PipeTransform {
  transform(value: unknown, ...args: unknown[]): unknown {
    return null;
  }
}
```

19. Write a code where you have to share data from the Parent to Child Component?

Le décorateur **@Input** permet d'envoyer toutes les données du parent à l'enfant.

```
// parent component
import { Component } from '@angular/core';
@Component({
  selector: 'app-parent',
  template: `
<app-child [childMessage]="parentMessage"></app-child>
`,
  styleUrls: ['./parent.component.css']
})
export class ParentComponent {
  parentMessage = "message from parent"
  constructor() { }
}

// child component
import { Component, Input } from '@angular/core';
@Component({
  selector: 'app-child',
  template: `Say {{ childMessage }}`,
  styleUrls: ['./child.component.css']
})
export class ChildComponent {
  @Input() childMessage: string;
  constructor() { }
}
```

20. Create a TypeScript class with a constructor and a function.

```
class IB {
  name: string;
  constructor(message: string) {
    this.name = message;
  }
  greet() {
    return "Hello, " + this.name + "How are you";
  }
}
let msg = new IB("IB");
```

Angular Interview Questions for Experienced

21. Angular by default, uses client-side rendering for its applications.

Angular fournit une technologie appelée **Angular Universal**, qui peut être utilisée pour rendre des applications côté serveur.

Les avantages d'utiliser Angular Universal sont :

- Les nouveaux utilisateurs peuvent voir instantanément une vue de l'application. Cela permet d'offrir une **meilleure expérience utilisateur**.
- De nombreux moteurs de recherche attendent des pages en HTML brut, ainsi, Universal peut s'assurer que votre contenu est disponible sur chaque moteur de recherche, ce qui conduit à **un meilleur référencement**.
- Toute application rendue côté serveur se **charge plus rapidement** puisque les pages rendues sont disponibles plus tôt pour le navigateur.

22. What is Eager and Lazy loading?

- **Loading:** la technique de chargement hâtif est la stratégie de chargement de module par défaut. Les modules de fonctionnalités de chargement rapide sont chargés avant le début du programme. Ceci est principalement utilisé pour les applications à petite échelle.
- **Lazy Loading:** le chargement paresseux charge les modules de fonctionnalités de manière dynamique selon les besoins. Cela accélère l'application. Il est utilisé pour les grands projets où tous les modules ne sont pas requis au départ.

23. What is view encapsulation in Angular?

L'encapsulation de la vue spécifie si le modèle et les styles du composant peuvent avoir un impact sur l'ensemble du programme ou vice versa.

Angular propose trois méthodes d'encapsulation :

- **Native:** le composant n'hérite pas des styles du code HTML principal. Les styles définis dans le décorateur @Component de ce composant ne s'appliquent qu'à ce composant.
- **Emulated (par défaut) :** le composant hérite des styles du code HTML principal. Les styles définis dans le décorateur @Component ne s'appliquent qu'à ce composant.
- **Aucun :** les styles du composant sont propagés vers le code HTML principal et sont donc accessibles à tous les composants de la page. Méfiez-vous des programmes qui ont des composants Aucun et Natif. Les styles seront répétés dans tous les composants avec une encapsulation native s'ils n'ont pas d'encapsulation.

24. What are RxJs in Angular ?

Reactive Extensions for JavaScript : Il est utilisé pour permettre l'utilisation d'observables dans notre projet JavaScript, nous permettant de faire de la programmation réactive. RxJS est utilisé dans de nombreux frameworks populaires,

La plupart du temps, rxJs est utilisé dans les appels HTTP avec angular. Comme les flux http sont des données asynchrones, nous pouvons nous y abonner et leur appliquer des filtres.

```
let stream1 = httpc.get("https://www.example.com/somedata");
let stream2 = stream1.pipe(filter(x=>x>3));
stream2.subscribe(res=>this.Success(res),res=>this.Error(res));
```

25. Explain string interpolation and property binding in Angular.

- La liaison de données est une fonctionnalité d'angular, qui fournit un moyen de communiquer entre le composant (modèle) et sa vue (modèle HTML).
- L'interpolation de chaîne utilise les doubles accolades `{{ }}` pour afficher les données du composant. Angular exécute automatiquement l'expression écrite à l'intérieur des accolades, par exemple, `{{ 2 + 2 }}` sera évalué par Angular et la sortie 4 sera affichée dans le modèle HTML. En utilisant la liaison de propriété, nous pouvons lier les propriétés DOM d'un élément HTML à la propriété d'un composant. La liaison de propriété utilise la syntaxe entre crochets `[ ]`.

26. How are observables different from promises?

The first difference is that an Observable is **lazy** whereas a Promise is **eager**.

Promise	Observable
Emits a single value	Emits multiple values over a period of time
Not Lazy	Lazy. An observable is not called until we subscribe to the observable
Cannot be cancelled	Can be cancelled by using the unsubscribe() method
	Observable provides operators like map, forEach, filter, reduce, retry, retryWhen etc.

Considérez l'observable suivant :

```
const observable = rxjs.Observable.create(observer => {
  console.log('Text inside an observable');
  observer.next('Hello world!');
  observer.complete();
});
console.log('Before subscribing an Observable');
observable.subscribe((message) => console.log(message));
```

Lorsque vous exécutez l'Observable ci-dessus, vous pouvez voir les messages affichés dans l'ordre suivant :

```
Before subscribing an Observable
Text inside an observable
Hello world!
```

Comme vous pouvez le voir, les observables sont paresseux. Observable ne s'exécute que lorsque quelqu'un s'abonne, par conséquent, le message "Avant de s'abonner..." s'affiche devant le message à l'intérieur de l'observable.

Considérons maintenant une Promesse :

```
const promise = new Promise((resolve, reject) => {
  console.log('Text inside promise');
  resolve('Hello world!');
});
console.log('Before calling then method on Promise');
greetingPoster.then(message => console.log(message));
```

En exécutant la promesse ci-dessus, les messages seront affichés dans l'ordre suivant :

```
Text inside promise
Before calling then method on Promise
Hello world!
```

Comme vous pouvez le voir, le message à l'intérieur de Promise s'affiche en premier. Cela signifie qu'une promesse s'exécute avant que la méthode **then** ne soit appelée. Par conséquent, les promesses sont **avidées**.

La différence suivante est que les promesses sont toujours **asynchrones**. Même lorsque la promesse est immédiatement résolue. Alors qu'un Observable peut être à la fois **synchrone** et **asynchrone**.

L'exemple ci-dessus d'un observable est le cas pour montrer qu'un observable est synchrone. Voyons le cas où un observable peut être asynchrone :

```
const observable = rxjs.Observable.create(observer => {
  setTimeout(() => {
    observer.next('Hello world!');
    observer.complete();
  }, 3000);
});
console.log('Before calling subscribe on an Observable');
observable.subscribe((data) => console.log(data));
console.log('After calling subscribe on an Observable');
```

Les messages seront affichés dans l'ordre suivant :

```
Before calling subscribe on an Observable
After calling subscribe on an Observable
Hello world!
```

Vous pouvez voir dans ce cas, observable s'exécute de manière asynchrone.

La différence suivante est que Observables peut émettre **plusieurs** valeurs alors que Promises ne peut émettre qu'une seule valeur.

La principale caractéristique de l'utilisation d'observables est l'utilisation d'**opérateurs**. Nous pouvons utiliser plusieurs opérateurs sur un observable alors qu'il n'y a pas une telle fonctionnalité dans une promesse.

27. Explain the concept of Dependency Injection?

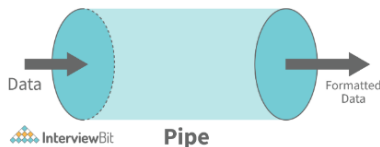
Les dépendances dans angular ne sont que des services qui ont des fonctionnalités. La fonctionnalité d'un service peut être requise par divers composants et directives dans une application. Angular fournit un mécanisme fluide par lequel nous pouvons injecter ces dépendances dans nos composants et nos directives.

28. What are pipes in Angular explain with an example?

Les pipes sont des fonctions qui simplifient le processus de câblage de vos expressions JavaScript et de leur transformation en la sortie souhaitée. Elles peuvent être comparées, par exemple, aux fonctions de chaîne dans d'autres langages de programmation.

Par exemple:

```
var css = myTheme.color | "red" ;
```



Exemple:

```
import { Component } from '@angular/core';

@Component({
  selector: 'app-root',
  template: `{{ title | uppercase }}`,
  styleUrls: ['./app.component.css']
})
export class AppComponent {
  title = 'this is an example of custom pies in angular';
}
```

29. What exactly is a parameterized pipe?

CECI EST UN EXEMPLE DE TUYAUX PERSONNALISÉS EN ANGULAIRE

Le pipe paramétré est construit en indiquant d'abord le nom du canal suivi de deux-points (:), puis de la valeur du paramètre. Si le tube prend en charge plusieurs arguments, utilisez des deux-points pour séparer les valeurs.

Exemple : Regardons un anniversaire avec un certain format (jj/MM/aaaa) :

```
import { Component } from '@angular/core';

@Component({
  selector: 'app-example',
  template: `<p>Birthday is {{ birthday | date:'dd/MM/yyyy' }}</p>`
})
export class ExampleComponent {
  birthday = new Date(2000, 7, 15);
}
```

30. What are class decorators?

Les décorateurs de classe sont les décorateurs de plus haut niveau qui déterminent le but des classes. Ils indiquent à Angular qu'une classe spécifique est un composant ou un module. Et le décorateur nous permet de déclarer cet effet sans avoir à écrire de code dans la classe.

Exemple:

```
import { NgModule, Component } from '@angular/core';

@Component({
  selector: 'class-component',
  template: '<div> This is a class component ! </div>',
})
export class ClassComponent {
  constructor() {
    console.log('This is a class component!');
  }
}

@NgModule({
  imports: [],
  declarations: [],
})
export class ClassModule {
  constructor() {
    console.log('This is a class module!');
  }
}
```

31. What are Method decorators?

Les décorateurs de méthodes, comme leur nom l'indique, sont utilisés pour ajouter des fonctionnalités aux méthodes définies dans notre classe.

Exemple : `@HostListener`, est un bon exemple de décorateurs de méthode.

```
import { Component, HostListener } from '@angular/core';
@Component({
  selector: 'method-component',
  template: '<div> This is a test method component ! </div>',
})
export class MethodComponent {
  @HostListener('click', ['$event'])
  onClick(event: Event) {
    console.log('clicked now this event is available !');
  }
}
```

32. What are property decorators?

Ce sont les deuxièmes types de décorateurs les plus populaires. Ils nous permettent de valoriser certaines propriétés de nos classes. Nous pouvons certainement comprendre pourquoi nous utilisons une certaine propriété de classe en utilisant un décorateur de propriété.

Il existe de nombreux décorateurs de propriétés disponibles, par exemple `@Input()`, `@Output`, `@ReadOnly()`, `@Override()`

```
import { Component, Input } from '@angular/core';
@Component({
  selector: 'prop-component',
  template: '<div> This is a test component ! </div>'
})
export class PropComponent {
  @Input()
  exampleProperty: string;
}
```

33. What is the Component Decorator in Angular?

L'objet de métadonnées reçu par le décorateur a des valeurs telles que `templateUrl`, `selector` et autres, avec la propriété `templateUrl` pointant vers un fichier HTML qui définit ce que vous voyez sur l'application.

```
import {Component} from '@angular/core';

@Component({
  selector: 'app-root',
  templateUrl: './app.component.html',
  styleUrls: ['./app.component.css']
})
export class AppComponent {
  title = 'Example component';
}
```

34. What are lifecycle hooks in Angular? Explain a few lifecycle hooks.

Every component in Angular has a lifecycle, and different phases it goes through from the time of creation to the time it's destroyed. Angular provides **hooks** to tap into these phases and trigger changes at specific phases in a lifecycle.

- **ngOnChanges()** Ce crochet/méthode est appelé avant `ngOnInit` et chaque fois qu'une ou plusieurs propriétés d'entrée du composant changent. Cette méthode/hook reçoit un objet `SimpleChanges` qui contient les valeurs précédentes et actuelles de la propriété.
- **ngOnInit()** Ce crochet est appelé une fois, après le crochet `ngOnChanges`. Il initialise le composant et définit les propriétés d'entrée du composant.

Constructor

ngOnChanges

ngOnInit

ngDoCheck

ngAfterContentInit

ngAfterContentChecked

ngAfterViewInit

ngAfterViewChecked

ngOnDestroy

- **ngDoCheck()** Il est appelé après **ngOnChanges** et **ngOnInit** et est utilisé pour détecter et agir sur les changements qui ne peuvent pas être détectés par Angular.
Nous pouvons implémenter notre algorithme de détection de changement dans ce crochet. **ngAfterContentInit()** Il est appelé après le premier crochet **ngDoCheck**. Ce hook répond une fois que le contenu est projeté à l'intérieur du composant.
- **ngAfterContentChecked()** Il est appelé après **ngAfterContentInit** et chaque **ngDoCheck** suivant. Il répond après vérification du contenu projeté.
- **ngAfterViewInit()** Il répond après l'initialisation de la vue d'un composant ou de la vue d'un composant enfant.
- **ngAfterViewChecked()** Il est appelé après **ngAfterViewInit** et il répond après que la vue du composant ou la vue du composant enfant a été vérifiée.
- **ngOnDestroy()** Il est appelé juste avant qu'Angular ne détruise le composant. Ce hook peut être utilisé pour nettoyer le code et détacher les gestionnaires d'événements

35. What are router links?

RouterLink est une directive de balise d'ancrage qui donne au routeur l'autorité sur ces éléments. Parce que les routes de navigation sont définies.

36. What exactly is the router state?

RouterState est une arborescence de routes. Les nœuds de cet arbre sont conscients des segments d'URL "consommés", des arguments récupérés et des données traitées. Vous pouvez utiliser le service Router et la propriété **routerState** pour obtenir le RouterState actuel à partir de n'importe où dans l'application.

```
@Component({templateUrl: 'example.html'})
class MyComponent {
  constructor(router: Router) {
    const state: RouterState = router.routerState;
    const root: ActivatedRoute = state.root;
    const child = root.firstChild;
    const id: Observable<string> = child.params.map(p => p.id);
    //...
  }
}
```

37. What does Angular Material means?

Angular Material est un package de composants d'interface utilisateur qui permet aux professionnels de créer des sites Web, des pages Web et des applications Web uniformes, attrayants et pleinement fonctionnels

38. What is ngOnInit?

ngOnInit est un **hook** de cycle de vie et une fonction de rappel utilisés par Angular pour marquer la création d'un composant. Il n'accepte aucun argument et renvoie un type void.

39. What is transpiling in Angular ?

Transpiling est le processus de transformation du code source d'un langage de programmation dans le code source d'un autre. En règle générale, dans Angular, cela implique de traduire TypeScript en JavaScript. TypeScript (ou un autre langage comme Dart) peut être utilisé pour développer du code pour votre application Angular, qui est ensuite transpilé en JavaScript. Cela se produit naturellement et en interne

40. What are HTTP interceptors ?

En utilisant le HttpClient, les **intercepteurs** nous permettent d'intercepter les requêtes HTTP entrantes et sortantes. Ils sont capables de gérer à la fois HttpRequest et HttpResponse. Nous pouvons modifier ou mettre à jour la valeur de la requête en interceptant la requête HTTP, et nous pouvons effectuer certaines actions spécifiées sur un code d'état ou un message spécifique en interceptant la réponse.

Exemple : Dans l'exemple suivant, nous allons définir le support d'en-tête d'autorisation pour toutes les requêtes :

```
token.interceptor.ts
import { Injectable } from '@angular/core';
import { HttpInterceptor, HttpRequest, HttpHandler, HttpEvent } from '@angular/common/http';
import { Observable } from 'rxjs/Observable';

@Injectable()
export class TokenInterceptor implements HttpInterceptor {
  public intercept(req: HttpRequest<any>, next: HttpHandler): Observable<HttpEvent<any>> {
    const token = localStorage.getItem('token') as string;
    if (token) {
      req = req.clone({
        setHeaders: {
          'Authorization': `Bearer ${token}`
        }
      });
    }
    return next.handle(req);
  }
}
```

1

Nous devons enregistrer l'intercepteur en tant que singleton dans les fournisseurs de modules

```
app.module.ts
import { NgModule } from '@angular/core';
import { BrowserModule } from '@angular/platform-browser';
import { HTTP_INTERCEPTORS } from '@angular/common/http';
import { AppComponent } from './app.component';
import { TokenInterceptor } from './token.interceptor';

@NgModule({
  imports: [
    BrowserModule
  ],
  declarations: [
    AppComponent
  ],
  bootstrap: [AppComponent],
  providers: [{
    provide: HTTP_INTERCEPTORS,
    useClass: TokenInterceptor,
    multi: true
  }]
})
export class AppModule {}
```

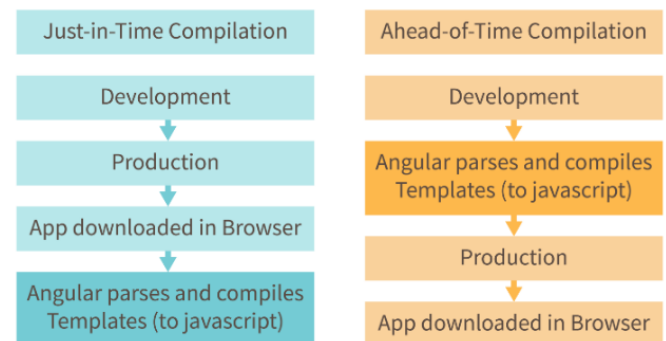
2

41. What is AOT compilation? What are the advantages of AOT?

Chaque application Angular se compose de composants et de modèles que le navigateur ne peut pas comprendre. Par conséquent, toutes les applications angulaires doivent d'abord être compilées avant de s'exécuter dans le navigateur.

Angular propose deux types de compilation :

- **JIT(Just-in-Time) compilation**
- **AOT(Ahead-of-Time) compilation**



Dans la compilation JIT, l'application se compile à l'intérieur du navigateur pendant l'exécution. Alors que dans la compilation AOT, l'application se compile pendant le temps de construction.

Les avantages de l'utilisation de la compilation AOT sont :

- Étant donné que l'application se compile avant de s'exécuter dans le navigateur, le navigateur charge le code exécutable et restitue l'application immédiatement, ce qui **accélère le rendu** .
- Dans la compilation AOT, le compilateur envoie les fichiers HTML et CSS externes avec l'application, éliminant les requêtes AJAX séparées pour ces fichiers source, ce qui réduit le **nombre de requêtes ajax** .
- Les développeurs peuvent détecter et gérer les erreurs pendant la phase de construction, ce qui aide à **minimiser les erreurs** .
- Le compilateur AOT ajoute du HTML et des modèles dans les fichiers JS avant qu'ils ne s'exécutent dans le navigateur. Pour cette raison, il n'y a pas de fichiers HTML supplémentaires à lire, ce qui offre une **meilleure sécurité** à l'application.

Par défaut, angular construit et sert l'application à l'aide du compilateur JIT :

```
ng build
ng serve
```

Pour utiliser le compilateur AOT, les modifications suivantes doivent être apportées :

```
ng build --aot
ng serve --aot
```

42. What is Change Detection, and how does the Change Detection Mechanism work?

Le processus de synchronisation d'un modèle avec une vue est appelé détection de changement. Même lors de l'utilisation du modèle ng pour implémenter une liaison bidirectionnelle, qui est du sucre syntaxique au-dessus d'un flux unidirectionnel

43. What is a bootstrapping module?

Chaque application contient au moins un module Angular, appelé module d'amorçage. AppModule est le nom le plus populaire pour cela.

Exemple : Voici la structure par défaut d'un AppModule généré par AngularCLI :

```
import { BrowserModule } from '@angular/platform-browser';
import { NgModule } from '@angular/core';
import { FormsModule } from '@angular/forms';
import { HttpClientModule } from '@angular/common/http';

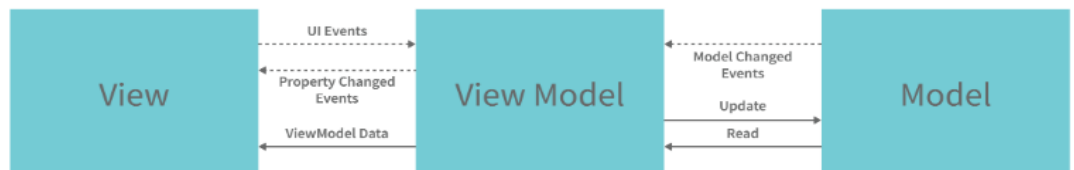
import { AppComponent } from './app.component';

@NgModule({
  declarations: [
    AppComponent
  ],
  imports: [
    BrowserModule,
    FormsModule,
    HttpClientModule
  ],
  providers: [],
  bootstrap: [AppComponent]
})
export class AppModule { }
```

44. Explain MVVM architecture

MVVM architecture consists of three parts:

1. Model
2. View
3. ViewModel



- **Le modèle** contient la structure d'une entité. En termes simples, il contient les données d'un objet.
- **La vue** est la couche visuelle de l'application. Il affiche les données contenues dans le modèle. En termes angulaires, ce sera le modèle HTML d'un composant.
- **ViewModel** est une couche abstraite de l'application. Un modèle de vue gère la logique de l'application. Il gère les données d'un modèle et les affiche dans la vue.

View et ViewModel sont connectés avec la liaison de données (liaison de données bidirectionnelle dans ce cas). Tout changement dans la vue, le modèle de vue prend une note et modifie les données appropriées à l'intérieur du modèle.

Angular Scenario Based Interview Questions

45. How do you choose an element from a component template?

Pour accéder directement aux éléments de la vue, utilisez la directive **@ViewChild**. Considérez un élément d'entrée avec une référence.

```
<input #example>
```

et construire une directive enfant de vue accessible dans le crochet de cycle de vie `ngAfterViewInit`

```
@ViewChild('example') input;

ngAfterViewInit() {
  console.log(this.input.nativeElement.value);
}
```

46. How do you deal with errors in observables?

Au lieu de dépendre de `try/catch`, qui est inutile dans un contexte asynchrone, vous pouvez gérer les problèmes en définissant un rappel d'erreur sur l'observateur.

Exemple : Vous pouvez créer un rappel d'erreur comme indiqué ci-dessous.

```
myObservable.subscribe({
  next(number) { console.log('Next number: ' + number)},
  error(err) { console.log('An error received ' + err)}
});
```

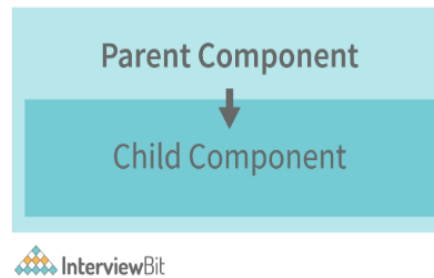
47. How can I include SASS into an Angular project?

```
ng new <Your_Project_Name> --style = sass
```

Si vous souhaitez modifier le style de votre projet, utilisez la commande `ng set`.

```
ng set defaults.styleExt scss
```

48. How does one share data between components in Angular?



InterviewBit

- Parent à enfant utilisant le décorateur `@Input`

Considérez le composant parent suivant :

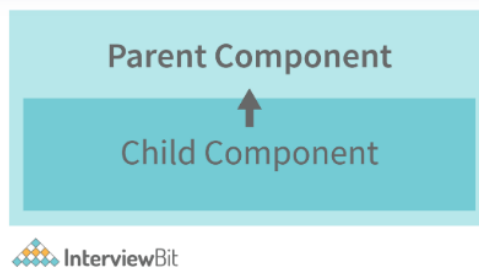
```
@Component({
  selector: 'app-parent',
  template: `
    <app-child [data]=data></app-child>
  `,
  styleUrls: ['./parent.component.css']
})
export class ParentComponent {
  data:string = "Message from parent";
  constructor() { }
}
```

Dans le composant parent ci-dessus, nous transmettons la propriété "data" au composant enfant suivant :

```
import { Component, Input } from '@angular/core';

@Component({
  selector: 'app-child',
  template: `
    <p>{{data}}</p>
  `,
  styleUrls: ['./child.component.css']
})
export class ChildComponent {
  @Input() data:string
  constructor() { }
}
```

Dans le composant enfant, nous utilisons le décorateur `@Input` pour capturer les données provenant d'un composant parent et les utiliser dans le modèle du composant enfant.



- **Enfant à parent utilisant le décorateur @ViewChild**
- Composant enfant :**

```

import {Component} from '@angular/core';
@Component({
  selector: 'app-child',
  template: `
    <p>{{data}}</p>
  `,
  styleUrls: ['./child.component.css']
})
export class ChildComponent {
  data:string = "Message from child to parent";
  constructor() { }
}
  
```

Composant parent

```

import { Component,ViewChild, AfterViewInit} from '@angular/core';
import { ChildComponent } from '../child/child.component';
@Component({
  selector: 'app-parent',
  template: `
    <p>{{dataFromChild}}</p>
  `,
  styleUrls: ['./parent.component.css']
})
export class ParentComponent implements AfterViewInit {
  dataFromChild: string;
  @ViewChild(ChildComponent,{static:false}) child;
  ngAfterViewInit(){
    this.dataFromChild = this.child.data;
  }
  constructor() { }
}
  
```

Dans l'exemple ci-dessus, une propriété nommée "data" est transmise du composant enfant au composant parent.

Le décorateur **@ViewChild** est utilisé pour référencer le composant enfant en tant que propriété "enfant".

À l'aide du crochet **ngAfterViewInit**, nous attribuons la propriété data de l'enfant à la propriété messageFromChild et l'utilisons dans le modèle du composant parent.

- **Enfant à parent utilisant @Output et EventEmitter**

Dans cette méthode, nous lions un élément DOM à l'intérieur du composant enfant, à un événement (événement **click** par exemple) et en utilisant cet événement nous émettons des données qui seront capturées par le composant parent :

Composant enfant :

```

import {Component, Output, EventEmitter} from '@angular/core';
@Component({
  selector: 'app-child',
  template: `
    <button (click)="emitData()">Click to emit data</button>
  `,
  styleUrls: ['./child.component.css']
})
export class ChildComponent {
  data:string = "Message from child to parent";
  @Output() dataEvent = new EventEmitter<string>();
  constructor() { }
  emitData(){
    this.dataEvent.emit(this.data);
  }
}
  
```

Comme vous pouvez le voir dans le composant enfant, nous avons utilisé la propriété **@Output** pour lier un **EventEmitter**. Cet émetteur d'événements émet des données lorsque le bouton du modèle est cliqué.

Dans le modèle du composant parent, nous pouvons capturer les données émises comme ceci :

```

<app-child (dataEvent)="receiveData($event)"></app-child>
  
```

Ensuite, à l'intérieur de la fonction receiveData, nous pouvons gérer les données émises :

```

receiveData($event){
  this.dataFromChild = $event;
}
  
```